LABA LIBERA ACCADEMIA BELLE ARTI

Corso propedeutico di programmazione

Internet, API, linguaggi di programmazione e web

Andrea Cometti

Indice

1 Internet e reti **1**

1.1 Che cos'è Internet 1

1.2 Percorso di una richiesta 1

1.3 Indirizzi e nomi 1

2 API e comunicazione tra sistemi **1**

2.1 Che cos'è un'API 2

2.2 Autenticazione e autorizzazione 2

3 Linguaggi di programmazione **2**

3.1 General purpose e specifici di dominio 2

3.2 Livello di astrazione 2

3.3 Compilazione ed esecuzione 3

3.4 REPL 3

4 Tecnologie web: HTML, CSS e JavaScript **3**

4.1 HTML: struttura dei contenuti 3

4.2 CSS: aspetto grafico 4

4.3 JavaScript: comportamento e interattività 4

4.4 Come collaborano 4

5 Intelligenza artificiale e reti neurali **4**

5.1 Neuroni artificiali 5

5.2 Pesi e parametri 5

5.3 Quando un'AI usa le API? 5

1 Internet e reti

Prima di scrivere codice conviene capire dove, di fatto, quel codice vive e come comunica. Quasi ogni programma che useremo, direttamente o indirettamente, si appoggia a Internet: vediamo come funziona, partendo dalle basi.

1.1 Che cos'è Internet

Internet è una **rete di reti**: non è un'unica grande infrastruttura, ma l'insieme di tante reti di dispositivi sparse per il mondo, collegate tra loro in modo da potersi scambiare dati.

Per farlo, Internet usa la **comunicazione a pacchetto**. I dati non viaggiano in un unico blocco: vengono suddivisi in piccole unità, i *pacchetti*, che attraversano la rete e vengono ricomposti o elaborati a destinazione. Può capitare che pacchetti diversi della stessa comunicazione seguano percorsi differenti.

A mettere ordine in tutto questo c'è **TCP/IP**, una *suite* di protocolli, cioè di regole condivise che permettono a dispositivi anche molto diversi di capirsi. IP si occupa dell'indirizzamento e dell'inoltro dei pacchetti; TCP, quando viene utilizzato, aggiunge tra le altre cose una trasmissione affidabile e ordinata.

1.2 Percorso di una richiesta

Quando apriamo una pagina o usiamo un'app, la nostra richiesta compie un viaggio. In forma semplificata possiamo immaginarlo così:

$$\text{utente} \rightarrow \text{router} \rightarrow \text{Internet} \rightarrow \text{server} \rightarrow \text{risposta all'utente}$$

Si tratta appunto di uno schema semplificato: il percorso reale dipende dalla rete, dal servizio e dai protocolli coinvolti. In questo viaggio compare anche il **gateway**, un punto che collega reti o sistemi differenti. In una rete domestica, questo ruolo è spesso svolto proprio dal router.

1.3 Indirizzi e nomi

Perché una richiesta arrivi a destinazione, serve un modo per indicare *a chi* è destinata. Entrano in gioco tre concetti.

L'**indirizzo IP** è un indirizzo logico che identifica un'interfaccia di rete e permette di instradare i dati. Un dispositivo può avere più interfacce, e quindi anche più indirizzi.

Ricordare sequenze di numeri non è però comodo per le persone, ed è qui che interviene il **DNS**: un sistema che traduce i nomi di dominio, come `example.com`, in informazioni utili a raggiungere i servizi, tra cui gli indirizzi IP. È un po' la rubrica telefonica di Internet.

Infine **localhost**, un nome convenzionale che indica il dispositivo su cui stiamo lavorando. Di solito corrisponde agli indirizzi di loopback `127.0.0.1` (IPv4) e `::1` (IPv6). Attenzione a un equivoco frequente: localhost non è l'indirizzo pubblico del dispositivo, ma un modo per parlare con se stessi.

2 API e comunicazione tra sistemi

Dopo aver visto come viaggiano i dati, possiamo chiederci come fanno due programmi a chiedersi qualcosa a vicenda. La risposta, molto spesso, sono le API.

2.1 Che cos'è un'API

Un'**API** (*Application Programming Interface*) è un insieme di regole e strumenti con cui un software mette a disposizione funzioni o dati che altri software possono utilizzare. In pratica, un'API definisce *come* richiedere un servizio e *che tipo di risposta* aspettarsi. Alcune API sono locali, altre permettono la comunicazione attraverso una rete.

Un esempio concreto: un'app meteo non possiede le previsioni, ma può usare un'API per chiederle a un server e poi mostrare i risultati all'utente.

2.2 Autenticazione e autorizzazione

Quando un servizio è esposto tramite API, di solito vuole sapere *chi* lo sta usando e *cosa* gli è permesso fare. Sono due domande distinte, e vale la pena tenerle separate.

L'**autenticazione** è la verifica dell'identità di un utente o di un sistema: "sei davvero chi dici di essere?". L'**autorizzazione**, invece, stabilisce quali operazioni o risorse sono consentite a quell'utente o sistema: "cosa puoi fare, una volta riconosciuto?".

Per gestire questi aspetti si incontrano spesso due strumenti dai nomi simili, che però non vanno confusi:

- **API key**: una chiave usata per identificare un'applicazione o un progetto e, a seconda del servizio, controllarne l'accesso e l'utilizzo. Non equivale necessariamente a nome utente e password, e non va considerata sempre sufficiente per autenticare una persona.
- **Passkey**: una credenziale di accesso basata su crittografia a chiave pubblica, usata per autenticare un utente. Nonostante l'assonanza, in generale non è un tipo di API key.

3 Linguaggi di programmazione

Ora che sappiamo come i sistemi comunicano, passiamo a come si scrivono i programmi. I linguaggi di programmazione sono tanti, e possono essere classificati con criteri diversi.

3.1 General purpose e specifici di dominio

Un primo modo per distinguerli è guardare allo *scopo*. I linguaggi **general purpose** si possono usare per molti tipi di problemi e applicazioni: Python, Java, C e C++ ne sono esempi. I linguaggi **specifici di dominio** (*DSL, Domain-Specific Language*) sono invece progettati per un ambito più circoscritto: SQL, per esempio, serve a interrogare e gestire dati nei database, mentre altri linguaggi descrivono circuiti o controllano macchine.

Questa distinzione dipende dallo scopo e dal contesto, non dalla "forza" del linguaggio: un linguaggio specifico non è necessariamente meno potente, è semplicemente pensato per un insieme più limitato di compiti.

3.2 Livello di astrazione

Un altro criterio è il *livello di astrazione*, cioè quanto il linguaggio ci tiene lontani dai dettagli della macchina. I linguaggi **di alto livello** offrono costrutti più vicini al modo in cui le persone descrivono un problema e nascondono molti dettagli tecnici. I linguaggi e le istruzioni **di basso livello**, come il linguaggio macchina e l'assembly, sono invece più vicini all'architettura hardware.

C e C++ meritano una nota a parte: vengono generalmente classificati come linguaggi di alto livello, ma permettono anche un controllo dettagliato della memoria e operazioni molto vicine alla macchina.

A questo punto è utile fissare due termini che si confondono facilmente. Il **codice sorgente** è il testo che scriviamo in un linguaggio di programmazione; il **codice macchina** è la rappresentazione delle istruzioni eseguibili direttamente dal processore. Non sono sinonimi: il primo è pensato per le persone, il secondo per l'hardware.

3.3 Compilazione ed esecuzione

Se il codice sorgente non è eseguibile direttamente dal processore, come fa a diventare un programma in funzione? Ci sono due strategie principali.

La **compilazione** traduce il codice sorgente, o una sua parte, in una rappresentazione eseguibile o intermedia. L'**interpretazione**, invece, prevede un interprete che legge ed esegue le istruzioni, oppure una loro rappresentazione, man mano.

Spesso si sente parlare di linguaggi "compilati" e "interpretati" come di due categorie rigide, ma la distinzione non è assoluta: lo stesso linguaggio può essere eseguito con tecniche diverse. JavaScript, per esempio, può essere interpretato e allo stesso tempo compilato durante l'esecuzione tramite *JIT* (*Just-In-Time*).

Quanto alle **prestazioni**, la compilazione può migliorarle e consentire ottimizzazioni, ma la velocità e l'efficienza dipendono anche dal programma, dall'ambiente e dall'implementazione specifica: non basta sapere se un linguaggio è compilato per dire che sarà più veloce.

3.4 REPL

Quando si impara a programmare è molto utile poter provare un'istruzione e vederne subito l'effetto. Lo permette il **REPL**, acronimo di *Read–Eval–Print Loop*, che funziona in quattro tempi:

1. **Read**: legge un comando;
2. **Eval**: lo valuta o lo esegue;
3. **Print**: mostra il risultato;
4. **Loop**: ripete il processo.

Un REPL è disponibile in diversi linguaggi e non è una caratteristica esclusiva dei linguaggi interpretati.

4 Tecnologie web: HTML, CSS e JavaScript

Il web è il luogo in cui molti di questi concetti si incontrano. Una pagina web, anche semplice, nasce dalla collaborazione di tre tecnologie, ciascuna con un compito preciso.

4.1 HTML: struttura dei contenuti

HTML (*HyperText Markup Language*) è il linguaggio che organizza i contenuti di una pagina tramite elementi, comunemente indicati attraverso i tag. Va ricordato che HTML non è, in senso

stretto, un linguaggio di programmazione: descrive la struttura, non una sequenza di istruzioni da eseguire. Ecco un esempio minimo:

```
<!doctype html>
<html lang="it">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale
       =1">
    <title>La mia pagina</title>
  </head>
  <body>
    <h1>Ciao!</h1>
    <p>Questa e una pagina web.</p>
  </body>
</html>
```

4.2 CSS: aspetto grafico

Una pagina con la sola struttura è funzionale ma spoglia. A occuparsi dell'aspetto ci pensa **CSS** (*Cascading Style Sheets*), che descrive la presentazione degli elementi HTML: colori, caratteri, spaziature, dimensioni e disposizione. Le regole CSS si basano su selettori, che indicano a quali elementi applicarle, e su dichiarazioni formate da proprietà e valori.

4.3 JavaScript: comportamento e interattività

Per rendere la pagina viva serve **JavaScript**, un linguaggio di programmazione che permette di aggiungere comportamenti dinamici, reagire agli eventi (un clic, un tasto premuto) e modificare la pagina mentre è aperta. Non vive solo nel browser: può essere usato anche altrove, per esempio sul server, tramite ambienti di esecuzione dedicati.

4.4 Come collaborano

Riassumendo, i tre strumenti si dividono il lavoro:

- **HTML**: struttura e contenuti;
- **CSS**: presentazione grafica;
- **JavaScript**: comportamento e interattività.

Le applicazioni web moderne possono poi combinare queste tecnologie con altre librerie, strumenti e componenti compilati. Per questo, quando un'applicazione come Figma gira nel browser, non è corretto dedurre da questo solo fatto quale sia la sua tecnologia interna.

5 Intelligenza artificiale e reti neurali

Chiudiamo con un argomento che riprende molti dei temi visti finora: come funzionano, a grandi linee, i modelli di intelligenza artificiale, e in che modo un'applicazione può servirsene.

5.1 Neuroni artificiali

Un **neurone artificiale** è l'unità di calcolo di una rete neurale. Riceve dei valori numerici, li combina usando dei pesi e un eventuale termine di bias, applica una funzione e produce un risultato in uscita. Il nome richiama i neuroni biologici, ma l'analogia si ferma qui: non è un neurone biologico e, da solo, non rappresenta affatto "l'intelligenza" del sistema.

5.2 Pesi e parametri

I **pesi** (*weights*) sono valori numerici che stabiliscono quanto ciascun segnale in ingresso contribuisce al calcolo di un'unità della rete. Insieme ai bias, quando presenti, e ad altri valori apprendibili, formano i **parametri del modello**.

Durante l'addestramento il modello aggiorna questi parametri per ridurre una funzione di errore, detta anche funzione di perdita: è così che apprende schemi dai dati. Un punto importante è che i pesi non corrispondono necessariamente a singole informazioni o fatti memorizzati; contribuiscono piuttosto a rappresentare regolarità e relazioni apprese.

5.3 Quando un'AI usa le API?

Un'applicazione può chiamare un'API di intelligenza artificiale quando vuole inviare dei dati a un modello e ottenerne una risposta, senza dover eseguire il modello sul proprio dispositivo. Il flusso, in generale, è questo:

1. l'applicazione invia una richiesta al servizio, per esempio un testo o un'immagine;
2. il servizio elabora la richiesta con il modello;
3. il servizio restituisce una risposta, che l'applicazione usa per mostrare un risultato o compiere un'altra operazione.

Un esempio: un sito invia una domanda a un servizio AI tramite API e mostra all'utente la risposta generata. Qui conviene tenere distinti i ruoli: l'API è il mezzo di comunicazione, mentre il modello AI è il sistema che elabora la richiesta.